

# Some adversarial attacks on CNNs

## MATH 637 Final Project

Lauren Kimpel

University of Delaware

Fall 2025

# Table of Contents

1 Introduction and definitions

2 Implementations

# What is an adversarial example?

- **The Situation:**  Consider a classifier which takes in arbitrary user input in the form of image data and returns a label.
- In theory, anything that takes in this input is susceptible to ingesting and reasoning over bad data.
- I would like to submit an image that I've altered slightly so that the classifier mis-classifies the image. (Ex. if I've submitted a bowl of cereal, the model will return a label of "cat" instead of a bowl of cereal, etc.)
- I want to make the alteration visually undetectable.

These modified user inputs are referred to as **adversarial examples**.

This project focuses on results developed between two papers:

- Nicolas Papernot, Patrick McDaniel, Ian Goodfellow, Somesh Jha, Z. Berkay Celik, and Ananthram Swami. 2017. Practical Black-Box Attacks against Machine Learning. In Proceedings of the 2017 ACM on Asia Conference on Computer and Communications Security (ASIA CCS '17). Association for Computing Machinery, New York, NY, USA, 506–519. <https://doi.org/10.1145/3052973.3053009>
- Goodfellow, Ian J., Jonathon Shlens, and Christian Szegedy. "Explaining and harnessing adversarial examples." arXiv preprint arXiv:1412.6572 (2014).

# White Box approach: Fast Gradient Sign Method (FGSM)

- Computes an adversarial image by altering pixels based on some pre-defined magnitude according to the direction of the gradient
- Generally efficient (takes one step)

Given by

$$x_{\text{aug}} = x + \epsilon \cdot \text{sign}(\nabla_x(J(\theta, x, y_{\text{true}}))) \quad (1)$$

where

- $x$  is our chosen adversarial image
- $\theta$  is the model parameters
- $\epsilon$  is our pre-defined magnitude
- $J$  is the **classification loss function**
- $y_{\text{true}}$  is the “true” (accurate) label for the input image  $x$ .

# Justification for FGSM

- During training, gradient descent is used to minimize the loss function.
- Intuition is to move in the opposite direction of the gradient iteratively.
- FGSM flips this around so that instead of computing a **minimization** problem, we are now **maximizing** loss by backpropagating the gradient back to the input data to get  $\nabla_x J(\theta, x, y)$ , scaling a tad, and picking the direction which maximizes loss (given by sign.)
- **At test time**, depending on how moderate or obnoxious the modification parameter  $\epsilon$  is made to be, we are able to increase the likelihood of completely misclassifying an image, despite whether the model itself is generally accurate.
- This problem is only “interesting” if there is an added goal of undetectability. (We want small perturbations, but want to maximize the extent of the attack.)

# Goodfellow example

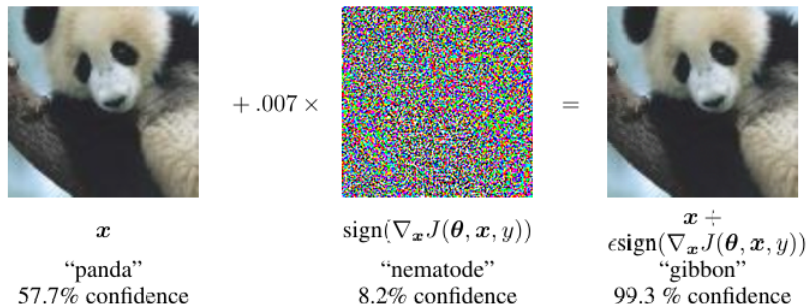


Figure: Adversarial example from Goodfellow et al, with  $\epsilon = 0.07$

# Some downsides...

- Requires knowledge of model's parameters and loss function
- In practice, an attacker will hardly be able to directly access the internals of the model
- This is commonly used as a proof-of-concept, and is thus referred to as a **white-box** attack.
- In a **black box attack**, the attacker will have little to no knowledge about the network at all! They must be a bit cleverer in order to approximate some useful information.



# “Black box” attacks

- Suppose Bob has left his image classification model  $V$  running remotely on his at-home Linux server that he hasn't configured properly to be invulnerable to outside perturbation. Anyone can submit an image and get a label back.
- Eve doesn't know anything about the parameters  $\theta$  of  $V$ .
- Eve is able to submit images to  $V$  and view its label.

# “Black box” attacks

- Eve chooses to construct her own CNN  $V'$  without knowing anything about  $V$ .
- Eve carefully picks image data so that  $V'$  can **train on the output of**  $V$ . There are many ways to do this; either she can continue to resubmit queries to  $V$  pulled from the dataset itself, or she can generate synthetic data using the output of  $V$ . This is referred to as **data augmentation**.
- Eve hopes for the best case scenario: that the “surrogate” model  $V'$  is functionally the same as  $V$ , and since she **does** have access to the parameters of  $V'$ , she now has access to estimations of the training weights and gradients of  $V$ .
- This concept is called **transferability**.

# Data Augmentation Step

I use **Jacobian-based data augmentation**, which produces synthetic points that (hopefully) serve as jumping-off points to learn the directions of the training points of the victim CNN.

Steps:

- Take the Jacobian matrix of the surrogate classifier  $J_F$
- Obtain the directions (signs in  $J_F$ ) with respect to our output data (labels)
- Augment by appending a **new direction** to the original samples:

$$x_{\text{aug}} = x + \lambda \cdot \text{sign}(J_F(y_{\text{out}})) \quad (2)$$

(seem familiar?)

For the attack implementation, I loosely followed a 2020-era(!!) tutorial developed by some Germans. (Their implementation uses Keras, and I simply will not do that. Hence the term “loose.”)<sup>1</sup>

<sup>1</sup>Tutorial link: <https://ai.hdm-stuttgart.de/news/2020/selected-topics-1-adversarial-attacks-3-black-box-attacks/>

# Big Picture (in MS Paint)

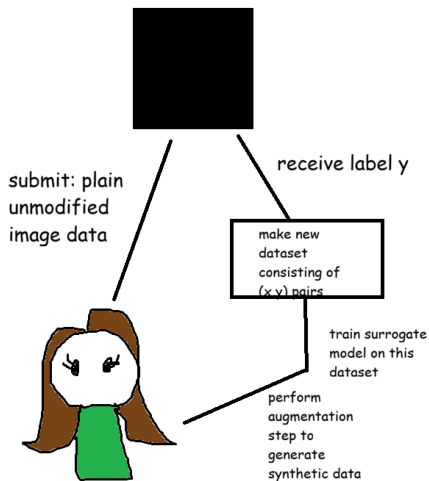


Figure: Main idea behind black box attacks

- **The goal of this project is to implement some common adversarial examples on CNNs we've seen in this course.**
- **Two methods are tested: FGSM (white box) and surrogate-victim with FGSM (black box).**

# Table of Contents

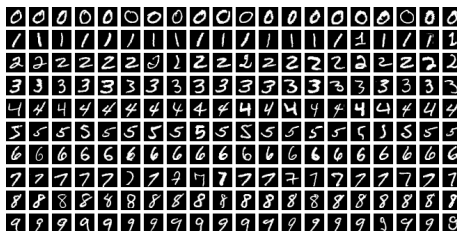
1 Introduction and definitions

2 Implementations

## White box implementation

# Implementation: White box FGSM for a simple CNN

- Code is written using PyTorch.
- **victim\_CNN:**
  - Train-test split is default (60,000 training images and 10,000 test images)
  - Two convolutions, ReLU as the activation function, and max-pooling
  - Opted for cross-entropy loss
- **Data:** MNIST dataset, consisting of hand-drawn numbers.<sup>2</sup>



**Figure:** Screenshot of some images in the MNIST dataset.

<sup>2</sup>(One could also hypothetically draw their own.)



```

def fgsm(loss_gradient, img_data, perturbation_parameter):
    """
    Takes in perturbation parameter epsilon and returns an
    altered image coordinate. This is a linear-style attack which
    can be applied to nonlinear data.
    """
    # get sign of loss gradient
    sign_loss_gradient = loss_gradient.sign()

    # pixel-wise adjustment
    new_img_data = img_data + perturbation_parameter * sign_loss_gradient

    # clipping
    new_img_data = torch.clamp(new_img_data, 0, 1)

    return new_img_data

def denorm(batch, mean=[0.1307], std=[0.3081]):
    if isinstance(mean, list):
        mean = torch.tensor(mean).to(device)
    if isinstance(std, list):
        std = torch.tensor(std).to(device)

    return batch * std.view(1, -1, 1, 1) + mean.view(1, -1, 1, 1)

```

Figure: FGSM function

# Results: White box

```
Iteration: 500. Loss: 0.24506519734859467. Accuracy: 90.12999725341797  
Iteration: 1000. Loss: 0.5003108382225037. Accuracy: 93.55999755859375
```

**Figure:** Base model accuracy is typically  $\approx 92$ -93.5% using 2 epochs with 500 training iterations per epoch (why did I make it those? This was an arbitrary decision, unfortunately.) I also used smaller batch sizes here.

```
Epsilon_val: 0. Loss: 0.02135918103158474. Accuracy: 92.05999755859375  
Epsilon_val: 0.01. Loss: 0.01724698208272457. Accuracy: 0.0944  
Epsilon_val: 0.05. Loss: 0.0003526780928950757. Accuracy: 0.0843  
Epsilon_val: 0.1. Loss: 0.03644658997654915. Accuracy: 0.0061
```

**Figure:** Even with slight perturbations, the model accuracy jumps down significantly (it's generally not supposed to be this aggressive; the reason is likely ~~lauren is incompetent~~ the implementation of the model)

# Nice explanatory graph

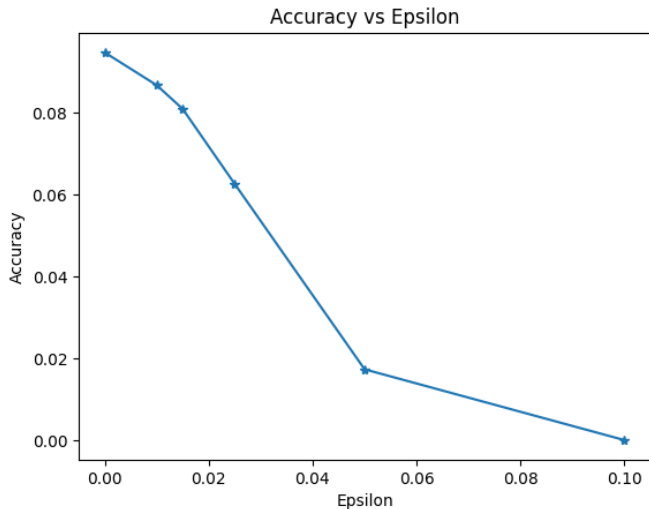


Figure: Literally everything from the previous slide but now we can see it

# Sanity check

To test the theory as to whether the model I used is particularly susceptible, I downloaded a model (LeNet) that has been trained on MNIST data, and ran the attack on that.

**Short answer:** Very well could be the fit of the model **IN ADDITION** to what choices we use for our  $\epsilon$  parameter.

```
Epsilon_val: 0. Loss: 2.204028844833374. Accuracy: 0.0751
Epsilon_val: 0.01. Loss: 2.1913962364196777. Accuracy: 0.0889
Epsilon_val: 0.1. Loss: 2.1945159435272217. Accuracy: 0.0827
Epsilon_val: 0.2. Loss: 2.228781223297119. Accuracy: 0.0458
Epsilon_val: 0.3. Loss: 2.17366886138916. Accuracy: 0.0287
Epsilon_val: 0.1. Loss: 2.1969590187072754. Accuracy: 0.0188
```

# LeNet Perturbation Results Visualized

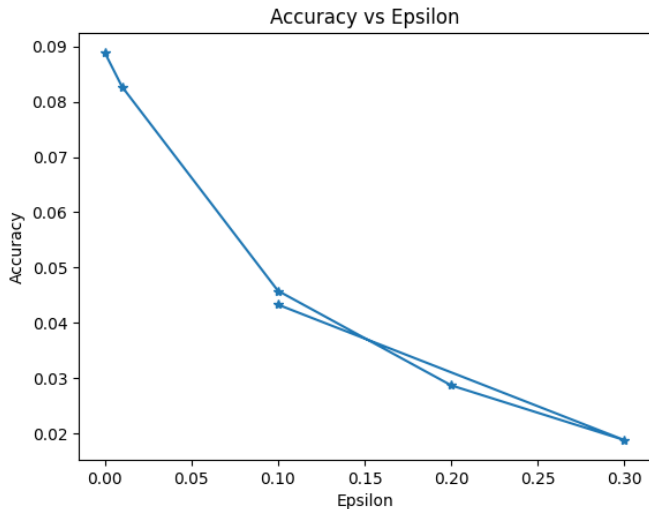


Figure: LeNet accuracy versus value of  $\epsilon$

## Black box implementation

# Implementation: black-box model for FGSM

- **victim\_CNN**: Same one as before.
- **Data augmentation**  $\lambda$ : 0.1 (default)
- **Data**: MNIST dataset (same as before.)
- **surrogate\_CNN**: a convolutional neural network used by Goodfellow et al. to demonstrate their black-box approach. Number of training epochs is actually a bit low (6!)
- For the initial batch of queries to the victim, I sampled  $N = 10$  images from the test dataset at random.

# Surrogate Model Implementation

For any interested parties:

```
class surrogate_CNN(nn.Module):
    """
    Link to paper:
    https://arxiv.org/pdf/1602.02697.pdf
    """
    def __init__(self):
        super(surrogate_CNN, self).__init__()
        self.cnn1 = nn.Conv2d(in_channels=1, out_channels=32, kernel_size=2, stride = 1, padding="same")
        self.maxpool1 = nn.MaxPool2d(kernel_size=2)
        self.cnn2 = nn.Conv2d(in_channels=32, out_channels=64, kernel_size=2, stride = 1, padding=0)
        self.maxpool2 = nn.MaxPool2d(kernel_size=2)
        self.flatten_dim = 64 * 6 * 6
        self.fc1 = nn.Linear(self.flatten_dim, 200)
        self.fc2 = nn.Linear(200, 200)
        self.fc3 = nn.Linear(200, 10)

    def forward(self, x):
        x = self.maxpool1(F.relu(self.cnn1(x)))
        x = self.maxpool2(F.relu(self.cnn2(x)))
        x = x.view(x.size(0), -1)
        x = F.relu(self.fc1(x))
        x = F.relu(self.fc2(x))
        x = self.fc3(x)
        output = F.softmax(x, dim=1)
        return output
```

Figure: Internals of the surrogate model.



# Results: Black box

- Hoo boy this was hard to implement<sup>TM</sup>
- For a truly black-box experience, I also ran this attack against LeNet and compared effectiveness. I'll focus on LeNet for the sake of time.

# Some images

```
Generating 10 adversarial samples.  
Iter 1/6  
  
Assigning labels from victim NN.  
  
Training step...  
  
...Done!  
Augmenting 10 samples...  
  
Iter 2/6  
  
Assigning labels from victim NN.  
  
Training step...  
  
...Done!  
Augmenting 20 samples...  
  
Iter 3/6
```

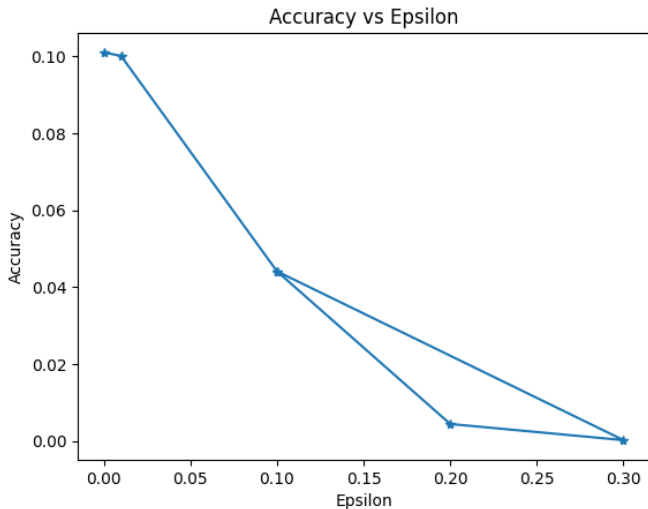
```
...Done!  
Finished training.  
Epsilon_val: 0. Loss: -0.1074218675494194. Accuracy: 0.101  
Epsilon_val: 0.01. Loss: -0.10622969269752502. Accuracy: 0.101  
Epsilon_val: 0.1. Loss: -0.1061868891119957. Accuracy: 0.1001  
Epsilon_val: 0.2. Loss: -0.10607395321130753. Accuracy: 0.0441  
Epsilon_val: 0.3. Loss: -0.10652890056371689. Accuracy: 0.0044  
Epsilon_val: 0.1. Loss: -0.10644304007291794. Accuracy: 0.0002
```

(a) Training loop for LeNet black-box

(b) Results relative to values of  $\epsilon$

Figure: Black box attempt with LeNet

# Another image



For these attacks to work, Eve needs to be mindful of the following:

- The training data format (the attack will be useless if I feed the network data that its architecture is not suited for)
- The  $\epsilon$ -sensitivity of the perturbations
  - We saw earlier that some networks are more reactive to higher or lower values of  $\epsilon$  when running the FGSM step.
- Possible mitigations:
  - **More robust loss functions** that penalize sensitivity to these modifications
  - **Include adversarial examples** in the dataset itself!
  - **Add noise to gradients**

**You are only as good as the data you train on.<sup>3</sup>**

---

<sup>3</sup>And also the person who implements the neural network, they have to know what they're doing a little.

- [https://docs.pytorch.org/tutorials/beginner/fgsm\\_tutorial.html](https://docs.pytorch.org/tutorials/beginner/fgsm_tutorial.html)
- <https://ai.hdm-stuttgart.de/news/2020/selected-topics-1-adversarial-attacks-3-black-box-attacks>
- Nicolas Papernot, Patrick McDaniel, Ian Goodfellow, Somesh Jha, Z. Berkay Celik, and Ananthram Swami. 2017. Practical Black-Box Attacks against Machine Learning. In Proceedings of the 2017 ACM on Asia Conference on Computer and Communications Security (ASIA CCS '17). Association for Computing Machinery, New York, NY, USA, 506–519. <https://doi.org/10.1145/3052973.3053009>
- Goodfellow, Ian J., Jonathon Shlens, and Christian Szegedy. "Explaining and harnessing adversarial examples." arXiv preprint arXiv:1412.6572 (2014).